

Assignment 2: A Real Life Competition - Hotel Ranking

Group 62

Anna De Martin (2906611), Alicja Dorobis (2746727), and David Žantovský (2869046)

Vrije Universiteit, Amsterdam 1081 HJ, Netherlands

Keywords: Data Mining · Recommender System · KNN · LightGBM

1 Introduction

This report describes our approach to a hotel properties ranking problem, using a dataset originally from a 2013 Kaggle competition [1]. The task requires ranking the hotel properties returned by a user's search query on Expedia in descending order of booking likelihood. We follow the CRISP-DM process: exploratory data analysis, data preparation and feature engineering, model training and evaluation, and finally a discussion of deployment and bias. Two models are trained and compared: a k-Nearest Neighbour (KNN) content-based recommender serving as a baseline, and a LightGBM LambdaMART ranker. Hotel booking platforms such as Expedia face the challenge of ranking hundreds of hotel options per search. Their algorithm's ability to correctly display relevant hotels directly determines the business's revenue. Recommender systems address this challenge.

2 Business Understanding and Related Work

Recommender systems can have many business applications. As Sarwar et al. [2] note, they help users find items they genuinely prefer while simultaneously increasing sales and customer retention for the platform. A hotel ranking model that accurately predicts booking probability, therefore serves both the user's need for relevant recommendations and the platform's commercial interest in maximising conversions.

Several works have tackled the problem of predicting hotel preferences and booking behaviour from search logs, including within the context of Expedia datasets. The most directly related body of work originates from the ICDM 2013 challenge, which used an Expedia dataset with identical features and the same relevance structure (bookings graded 5, clicks 1, and no interactions 0), evaluated using Normalised Discounted Cumulative Gain (NDCG). Liu et al. [3] describe the 5th-place solution. They first trained individual rankers (LambdaMART, Gradient Boosting, Random Forests, Factorisation Machines, and logistic regression) and combined them via an ensemble after normalising each

model’s score to achieve their highest performance. For their Gradient Boosting model, they noted that the most important features were related to location scores, price and ratings-derived signals. Another Expedia-based recommendation challenge involved predicting the top 5 hotel clusters out of 100, framing the task as ranked multiclass classification. Karakasoglu [4] and Mavalankar et al.[5] both found that ranking-aware methods, RankBoost in [4], and an ensemble of XGBoost, Random Forest, SGD, and Naive Bayes in [5], outperformed pointwise classifiers. Both studies identified geographic distance and destination ID as highly predictive features.

Xie and Lee [6] studied the same Expedia data provided for this assignment but applied an econometric approach. Using a Heckman two-stage probit model, they separated the click-through decision from the booking decision, demonstrating that the two are not independent. Their highly interpretable approach revealed that consumer-generated review scores are more predictive of bookings than industry star ratings, and that promotional incentives significantly increase conversion at both stages. Moreover, display position had a strong negative effect on click-through probability. This indicates the existence of position bias, which we also observed during our Exploratory Data Analysis. Wang [8] addresses this position bias in recommender systems directly, proposing a regularisation-based approach built on matrix factorisation. The method adds a penalty term to the loss function that encourages click counts across recommended positions to follow a uniform rather than a power-law distribution, thereby discouraging the model from exploiting position as a shortcut.

3 Exploratory Data Analysis and Data Preprocessing

3.1 Dataset Overview and Descriptive Statistics

The large size of the dataset made memory issues a salient problem. We downcast the column data types to conserve memory consumption. Columns such as `srch_id` and `prop_id` were cast to `int8` or `int16` with all float columns cast to `float32`. Hereby, we preserved the precision of these variables while freeing up memory to run the models.

The training set comprises 4,958,347 rows across 54 columns, spanning searches conducted from November 2012 to June 2013. Each row represents a single hotel displayed in response to a user’s search query. The dataset contains 199,795 unique queries (identified by `srch_id`) and 129,113 unique hotel properties (`prop_id`), with an average of approximately 24.8 hotels per search. The test set mirrors this structure, containing 4,959,183 rows across 50 columns, omitting `display position` and booking outcomes. Table 1 summarises the key descriptive statistics of these features. The statistics highlight the extreme sparsity of user history and competitor data, indicating the necessity of missing-value handling in our preprocessing pipeline.

The target variables of interest, clicks and bookings, exhibit an extremely low positive rate of just 4.47% and 2.79%, respectively. We also looked at inter-variable relationships, which revealed that first, three and four-star hotels are

Table 1: Descriptive Statistics of Numeric Features

Feature	Count	Mean	Std Dev	Min	Median	Max	Missing (%)
comp1_rate_percent_diff	94439	244.23	1165.51	2.00	10.00	30389.00	98.1
comp6_rate_percent_diff	96174	17.25	31.16	2.00	11.00	1620.00	98.1
comp1_rate	119930	0.48	0.64	-1.00	1.00	1.00	97.6
comp1_inv	129559	0.03	0.23	-1.00	0.00	1.00	97.4
comp4_rate_percent_diff	131086	175.32	5756.17	2.00	11.00	1001584.00	97.4
gross_bookings_usd	138390	386.28	821.17	0.00	218.40	159292.38	97.2
comp7_rate_percent_diff	138515	19.43	54.37	2.00	12.00	9900.00	97.2
comp6_rate	240157	0.13	0.56	-1.00	0.00	1.00	95.2
visitor_hist_adr_usd	252988	176.02	107.25	0.00	152.24	1958.70	94.9
visitor_hist_starrating	251866	3.37	0.69	1.41	3.45	5.00	94.9
comp6_inv	260976	0.08	0.30	-1.00	0.00	1.00	94.7
comp4_rate	307378	-0.02	0.58	-1.00	0.00	1.00	93.8
srch_query_affinity_score	317406	-24.15	15.74	-326.57	-20.45	-2.49	93.6
comp7_rate	315348	0.15	0.58	-1.00	0.00	1.00	93.6
comp4_inv	343663	0.10	0.34	-1.00	0.00	1.00	93.1
comp7_inv	356422	0.08	0.32	-1.00	0.00	1.00	92.8
comp3_rate_percent_diff	472797	27.07	1012.75	2.00	11.00	199266.00	90.5
comp2_rate_percent_diff	556238	18.49	239.90	2.00	11.00	168893.00	88.8
comp8_rate_percent_diff	614730	22.43	895.86	2.00	11.00	149400.00	87.6
comp5_rate_percent_diff	841099	29.85	1340.09	2.00	12.00	607561.00	83.0
comp3_rate	1534288	0.02	0.46	-1.00	0.00	1.00	69.1
comp3_inv	1650990	0.06	0.29	-1.00	0.00	1.00	66.7
comp8_rate	1916654	-0.06	0.47	-1.00	0.00	1.00	61.3
comp8_inv	1987503	0.01	0.20	-1.00	0.00	1.00	59.9
comp2_inv	2130269	0.04	0.23	-1.00	0.00	1.00	57.0
comp5_rate	2222373	0.02	0.52	-1.00	0.00	1.00	55.2
comp5_inv	2360020	0.05	0.27	-1.00	0.00	1.00	52.4
orig_destination_distance	3350565	1301.23	2018.45	0.01	386.60	11666.64	32.4
prop_location_score2	3867999	0.13	0.16	0.00	0.07	1.00	22.0
prop_review_score	4950983	3.78	1.03	0.00	4.00	5.00	0.1
srch_children_count	4958347	0.35	0.73	0.00	0.00	9.00	0.0
prop_starrating	4958347	3.18	1.05	0.00	3.00	5.00	0.0
prop_country_id	4958347	173.97	68.35	1.00	219.00	230.00	0.0
prop_id	4958347	70079.18	40609.92	1.00	69638.00	140821.00	0.0
visitor_location_country_id	4958347	175.34	65.92	1.00	219.00	231.00	0.0
site_id	4958347	9.95	7.65	1.00	5.00	34.00	0.0
srch_id	4958347	166366.56	96112.23	1.00	166507.00	332785.00	0.0
srch_adults_count	4958347	1.97	0.86	1.00	2.00	9.00	0.0
srch_booking_window	4958347	37.47	51.99	0.00	17.00	492.00	0.0
srch_length_of_stay	4958347	2.39	2.05	1.00	2.00	57.00	0.0
srch_destination_id	4958347	14042.63	8111.84	2.00	13541.00	28416.00	0.0
promotion_flag	4958347	0.22	0.41	0.00	0.00	1.00	0.0
price_usd	4958347	254.21	16000.44	0.00	122.00	19726328.00	0.0
position	4958347	16.86	10.43	1.00	16.00	40.00	0.0
prop_log_historical_price	4958347	4.32	1.83	0.00	4.91	6.21	0.0
prop_location_score1	4958347	2.87	1.53	0.00	2.77	6.98	0.0
prop_brand_bool	4958347	0.63	0.48	0.00	1.00	1.00	0.0
random_bool	4958347	0.30	0.46	0.00	0.00	1.00	0.0
srch_room_count	4958347	1.11	0.42	1.00	1.00	8.00	0.0
srch_saturday_night_bool	4958347	0.50	0.50	0.00	1.00	1.00	0.0
click_bool	4958347	0.04	0.21	0.00	0.00	1.00	0.0
booking_bool	4958347	0.03	0.16	0.00	0.00	1.00	0.0

clicked and booked more frequently than others (Figure 1a). Second, booking rates drop sharply as display position increases, especially for the non-random sort condition (`random_bool = 0`) (Figure 1c). Third, properties with an active promotion (`promotion_flag = 1`) tend to appear at a higher position, on average, compared to non-promoted hotels (average position of 14.8, compared to 17.4 for non-promoted hotels) (Figure 1b). Finally, we noticed that booked hotels cluster at noticeably lower price points than ignored properties, despite considerable overlap and a right-skewness in both distributions. Also, as reflected in Table 1, `price_usd` contains substantial outliers. Figure 1d value were therefore clipped at the 99th percentile for visualisation purposes.

Missing value rates vary dramatically across features. Our imputation strategy was different depending on the modelling techniques: LightGBM, LambdaMART and k-Nearest Neighbours (KNN). For the LightGBM model, we preserved missing values as `NaN` while generating explicit binary missingness flags (e.g., `has_review_score`). This allows the gradient boosting framework to natively route `NaN` values to the optimal child node during tree splitting, effectively

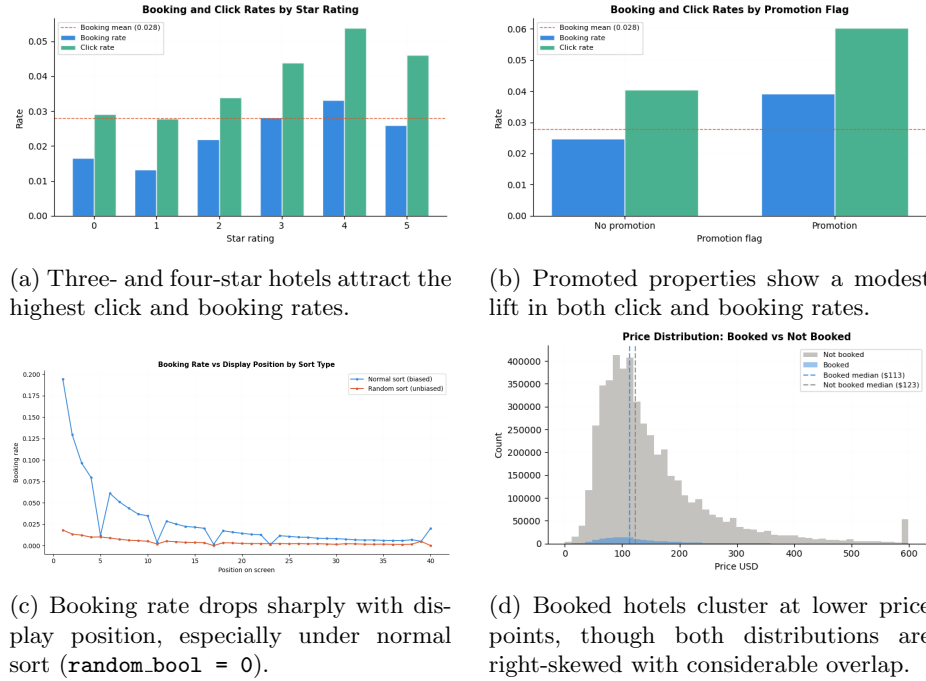


Fig. 1: Selected variable relationships with booking and click outcomes.

treating missingness as a distinct, informative category [20]. Conversely, because KNN distance metrics require complete numerical arrays, we filled missing values with zero for absolute counts or the destination-grouped median for scores.

Applying this framework to specific variables, we first addressed competitor signals (`comp1{8}`). Instead of imputing 24 sparse columns individually, we aggregated the data into two dense binary indicators denoting whether Expedia is cheaper (`is_cheapest_any`) or more expensive (`is_expensive_any`) than at least one competitor, subsequently dropping the original columns.

Visitor history variables (`visitor_hist_starrating` and `visitor_hist_adr_usd`) are missing for 94.9% of the dataset. Recognising that a null value indicates a user has no prior Expedia purchase history, we introduced a `new_customer` binary flag. A similar distinction was made for property review scores (`prop_review_score`), where a missing score denotes unavailable information rather than a zero rating. We generated a `has_review_score` flag. Underlying historical values were imputed with zero for KNN, whereas for LightGBM, they were left as NaN.

For the secondary location score (`prop_location_score2`), we filled its missing values with the destination-grouped median from the training set, alongside a corresponding missingness flag. Binary flags were similarly implemented for query affinity and physical distance, which were left as NaN for LightGBM, and imputed with zero and the median, respectively, for KNN. Finally, to rigorously prevent data leakage, the `gross_bookings_usd` column, which is observed only when a booking occurs, was dropped entirely before modeling.

3.2 Feature Engineering

Our feature engineering strategy was composed of two categories of new features, those being within search features and historical hotel performance features. The within search features were grouped using the `srch_id` column, while the hotel performance features were grouped using `prop_id`.

From variable `within_search_window`, we created two variables: `is_last_minute` based on the elbow (3.65 days) and `is_advanced_booking` (the median = 17 days).

Within Search Features We first chose to group by search query to capture how each hotel compares to the other hotels presented to the user in the same search. Liu et al. [3] used a similar "listwise feature" approach in their paper. The absolute value of a hotel's price means little without the context of what else is on offer, a same hotel at the same price may be expensive in one search and cheap in another. The rank within the search captures this context. Liu et al. [3] explicitly named `price_usd`, `prop_starrating`, and `prop_location_score2` as important features in their models' success. For the price of each hotel, we computed the dense rank within that search, the ratio of the hotel's price to the mean price of all hotels in one search, and the search level price statistics. We repeated this process for other quality metrics such as the star rating, the review scores, and the location scores. Furthermore, we computed features based on the users' booking history. The correlation matrix (Fig 2) of key features indicated that `visitor_hist_starrating` and `visitor_hist_adr_usd` were moderately positively correlated, showing that users that prefer higher star hotels also tend to pay more. We therefore computed the difference between the hotel's star rating and the user's historical star rating preference, and the difference between the hotel's price and the user's historical average daily rate. We then computed the discrepancy between a hotel's advertised star rating and review score as a proxy for quality misrepresentation. For example, if a hotel claimed to be 5 stars but was rated as 3 stars or if a hotel claimed to be 3 stars but was reviewed by users as 4 stars, it acts as a signal for booking likelihood. The idea being that users may be more likely to book hotels they see as trustworthy or delivering on their promise. Finally, we included interaction features that combined star rating and review score, as well as a weighted combination of both location scores.

Hotel Performance Features Our decision to group features by `prop_id` was largely based on the success others found in the original 2013 Kaggle competition [16] [1]. One of the top submissions contained a forum with participants discussing methods they used to improve their models. One such comment by Michael Jahrer cited that the average, standard deviation and median statistics of numeric features per `prop_id` contributed to their high score. Therefore, it may be that some hotels are consistently more popular than others regardless of who is searching. We computed historical booking and click rates per hotel and used total training appearances as a proxy for a hotel's popularity. Finally,

Table 2: Engineered features grouped by aggregation level

Group	Feature type	Statistics
srch_id	Price relative to search	Mean, Std, Ratio, Dense rank
srch_id	Quality ranks within search	Star rating, review, location scores
srch_id	User-hotel match	Star vs. history, price vs. history
srch_id	Quality interactions	Star $\times$ review, location combined
srch_id	Quality misrepresentation	Advertised star vs. review score
prop_id	Price history	Mean, Std, Median
prop_id	Booking & click rates	Mean
prop_id	Popularity	Appearance count
prop_id	Estimated position	Mean (inverted)
Derived Context & competitor flags		Binary
Derived Temporal & missingness flags		Binary

we looked at estimated position, the inverse of each hotel’s average historical display position, to identify the historical prominence of each hotel. A similar approach was also used in Liu et al.’s model where they computed click rates and booking rates per **prop_id** and per discrete price range, finding these historical performance aggregations to be among the most predictive features in their LambdaMART model [3].

Relevance Target We defined the target variable for the ranking model as **relevance**. Calculated using: $\text{relevance} = \text{booking_bool} \times 4 + \text{click_bool}$. Effectively yielding us three possible values: 0, 1, and 5. 0 meant a hotel was not interacted with at all, neither clicked nor booked. A 1 meant that a hotel was clicked on by the user and not booked. And 5 meant that a hotel was both clicked and booked by the user. We decided on this weighting based on the original competition’s evaluation framework [15].

4 Modeling and Evaluation

We used an 80/20 split on the dataset at the search level, meaning all rows belonging to a specific search query were assigned to the same split. This was done to mitigate the risk that would come with placing the hotel results from the same search into both the training and validation portions of the dataset, which may have led to artificially high scores when evaluating our models. With a total of 199,795 unique searches, 159,836 were used to train our models, and 39,959 were used to validate our models.

4.1 Evaluation Metric: NDCG@5

We evaluated our models using Normalized Discounted Cumulative Gain at position 5 (NDCG@5). This is a tailored metric for learning-to-rank tasks and the same metric used in the original 2013 Expedia Kaggle competition. The two core principles of NDCG are position and relevance. This means that the higher

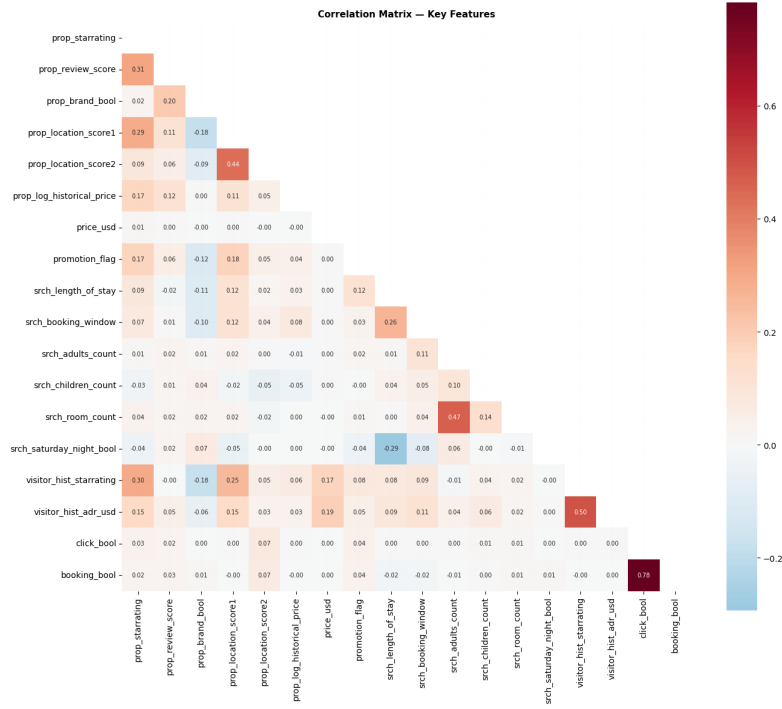


Fig. 2: Correlation Matrix of Key Features

an item appears on a ranked list, the more likely a user is to interact with it, and thereby the more impactful that item is. The NDCG favors models that rank more popularly booked hotels higher, and punishes models that rank these hotels lower [17].

The NDCG is calculated at K , which defines a cutoff point for ranked items in consideration, for our case $K = 5$. This means that only the top 5 ranked hotels per search contribute to the score. Then, a list of discounted cumulative gains (DCG) at K must be divided by the ideal DCG at K to result in the NDCG@ K [17].

4.2 K-Nearest Neighbours as Baseline

K-Nearest Neighbours is a non-parametric, instance-based learning algorithm. It makes predictions by identifying the k most similar training examples to a query point and aggregating their labels [11]. In this context, the algorithm identifies the k hotels in the training set with the most similar feature profiles based on distance metric. It then estimates booking probability as a weighted average of their relevance labels, where closer neighbours contribute proportionally more under distance weighting. To produce the final ranking, the hotels within each search are then ranked by descending predicted booking probability. Because hotels are compared based on shared features and past booking outcomes

to predict relevance, this implementation functions as an item-based similarity recommender system [2].

As the relationships between the data in this dataset were unlikely to be linear, we chose this algorithm for its non-parametric method, which makes no assumptions about the underlying data distribution. Additionally, this model was selected to serve as a baseline and for its direct interpretability [13].

Model training and hyperparameters The KNN model was implemented as a scikit-learn pipeline consisting of a feature scaling step followed by a KNeighborsClassifier. The scaler was placed inside the pipeline to prevent data leakage across cross-validation folds. A three-class classification formulation was used with relevance labels 0 (not interacted), 1 (clicked), and 2 (booked), and the ranking score was derived from the predicted class probabilities as $\text{score} = P(\text{booked}) + 0.1 \times P(\text{clicked})$, giving bookings ten times the weight of clicks consistent with their relative importance in the NDCG@5 objective.

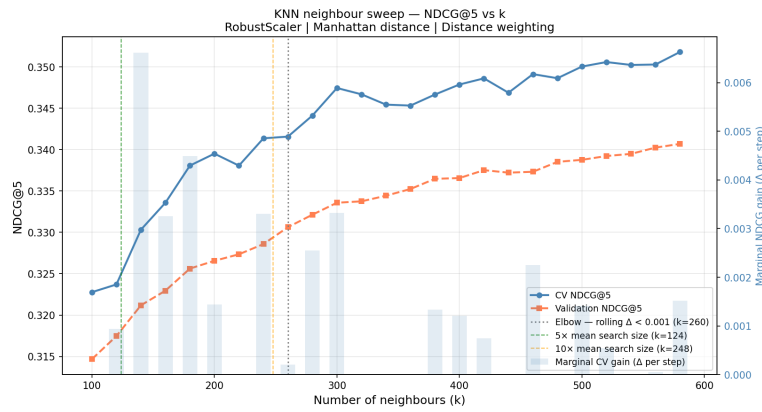


Fig. 3: Sweep over number of neighbours k

Firstly, the following hyperparameters were tuned via 5-fold cross-validation grid search: $k \in [40, 50, 65, 80, 90, 100, 300]$, StandardScaler and RobustScaler, Manhattan and Minkowski distance, distance and uniform weighting. To align the selection criterion with the ranking objective, the hyperparameters were evaluated on NDCG@5, not on classification accuracy. The sweep revealed the distance-based voting and RobustScaler as the optimal configuration, with the Manhattan distance selected over Minkowski. This is consistent with the finding of Aggarwal et al. [18] that the fractional L_1 norm (Manhattan) better concentrate distance mass in high-dimensional space, which makes them more discriminative than L_2 norm (Euclidean distance), when the number of features is large relative to the number of observations. Unsurprisingly, the distance-weighted voting was selected over uniform, as the closer neighbours contribute more to the booking probability than the farther ones. This reflects the intu-

ition that a highly similar hotel is a more informative reference point than a moderately similar one [11].

The best performance was achieved on maximum number of neighbours introduced in the initial sweep ($k = 300$). To minimise computation, the other parameters were fixed based on the initial sweep and the number of neighbours k was then selected through a dedicated sweep from $k = 100$ to $k = 580$ (see Fig 3. This analysis showed that NDCG@5 improves consistently with k , with the largest marginal gains occurring between $k=100$ and $k=260$, where the 3-point rolling mean of CV NDCG improvement averages 0.003–0.005 per 20-neighbour increment. Beyond 260, the gains fall below 0.001 per step, indicating diminishing returns. Thus, we selected $k = 400$ as the final optimal number, as it represents the point at which performance improvement is plateauing while avoiding unnecessary computational cost at inference time.

The final model achieved a cross-validation NDCG@5 of 0.3479 and a holdout validation NDCG@5 of 0.3366. The small gap between the two scores indicates that the model extrapolates well to unseen data without large overfitting. However, the overall performance of this algorithm reflects the inherent limitations of KNN when applied to a learning-to-rank task. Notably, KNN works as a pointwise approach, meaning it does not learn a ranking objective, but learns to predict whether the individual hotel will be booked. As Tie-Yan Liu notes, because the input object in a pointwise approach is a single document (here a hotel listing), the order is not visible in the learning process [19]. Thus, the algorithm does not have any concept of relative ordering within a search and cannot directly optimise the position-based metric (NDCG@5) used to evaluate it. Therefore, for our second model, we chose LightGBM, which learns the ranking itself.

4.3 LightGBM LambdaMART as Submission Model

We utilised LambdaMART, a listwise learning-to-rank algorithm that is directly optimised for NDCG, as our main model. This algorithm was implemented using the LightGBM library [20], a highly efficient gradient boosting framework where each tree corrects the errors of previous trees combined. LambdaMART treats ranking as a primary objective by computing lambda gradients that are weighted by the NDCG gain from swapping pairs of hotels within the same search. Hotels are therefore treated relative to each other, rather than being assigned independent scores to each hotel in isolation [23].

The model functions by assigning a stronger gradient signal for swaps that result in large changes in the NDCG score. It then combines these lambda gradients with Multiple Additive Regression Trees (MART). At each boosting iteration, a regression tree is trained to predict the lambda values based on our chosen hotel features and adds the output to the total ensemble of trees [24]. The final model is the sum of all the trees, producing our relevance target score for each hotel that is then used to sort the final rankings [23].

We found this technique when looking through successful submissions in the 2013 original Kaggle competition. The fifth-ranked submission by "Bing Xu, ML-

Rush and BrickMover” contains a forum discussion where multiple contributors reference using LambdaMART to achieve high scores [16].

Hyperparameters Optimization We implemented the model using LightGBM’s LGBMRanker with the lambdarank objective [20]. A $2 \times 2 \times 2$ grid search was conducted involving three hyperparameters (`learning_rate`, `num_leaves`, and `min_child_samples`) on a sample of 500,000 rows. Each hyperparameter configuration was evaluated on the validation set using NDCG@5 with early stopping (patience fo 150 rounds). We did not tune the number of trees directly; instead we tuned `n_estimators` to a ceiling of 10,000. The early stopping terminated training when the validation NDCG@5 score failed to improve for 250 consecutive rounds.

Table 3: LightGBM LambdaMART hyperparameter search space and optimal configuration

Parameter	Search space	Best value
<code>learning_rate</code>	{0.01, 0.05}	0.05
<code>num_leaves</code>	{127, 255}	255
<code>min_child_samples</code>	{50, 100}	50
Validation NDCG@5		0.4106
Kaggle NDCG@5		0.41058

We also included additional hyperparameters that we fixed. The L1 and L2 regularisation terms, at `reg_alpha=0.1` and `reg_lambda=1.0` respectively, were included because clicks and bookings are not perfect representations of true user preference. Here, L2 constrains weights to be small, while avoiding removing features entirely, thereby reducing sensitivity to noise [22]. We used L1 to push unimportant features towards zero to ensure our model remained sparse due to the large volume of features we were including [21]. We set `colsample_bytree`, the fraction of features sampled per tree, at 0.7 rather than the default of 1.0 to introduce more variation between trees in the ensemble and combat overfitting [20]. We also made sure to set `label_gain=[0,1,5]`, to reflect the values used in our relevance target formula, otherwise LightGBM set them to 0/1/2 equal steps thereby removing the weighting of booking versus click.

We found an optimal configuration of a `learning_rate` at 0.01, `num_leaves` at 255, and `min_child_samples` at 50, achieving a validation NDCG@5 of 0.4133. However, when applied to the full training set of approximately five million rows, this learning rate proved computationally expensive as each boosting iteration required substantially more computation. Given these constraints, we concluded on a `learning_rate` of 0.05 for the final model. Our choice of 255 `num_leaves` was made as it yielded the best results in the sweep and remained computationally manageable with regularization. The `min_child_samples` acts as the minimum number of training examples required in a leaf before splitting. We tested both 50

and 100, and decided on 50 as it consistently outperformed 100 across all sweep combinations on the ranking task. This configuration converged in approximately 241 iterations. The final model achieved a validation NDCG@5 of 0.4106 and a public Kaggle score of 0.41058, all results have been included in Table 2.

5 Deployment: Bias Mitigation

Algorithmic bias in hotel ranking systems carries direct business consequences, including like worse visibility and lower revenue for the hotels and worse predictions for the users. Consequently, we conducted a group fairness analysis focusing on two interconnected sources of bias, position bias and promotion bias, which can distort the genuine representation of a business’s quality.

5.1 Position and Promotion Bias

As demonstrated in our Exploratory Data Analysis (Figure 1c), booking rates drop sharply as display position increases. This effect is significantly more pronounced when Expedia chooses the display order (`random_bool = 0`) compared to a randomised sort (`random_bool = 1`). This indicates that hotels booked under the normal sort benefit not only from their intrinsic quality but also from their display order. This aligns with observations by Craswell et al., who note that while search engine click data reveals listing relevance, it is inherently skewed by presentation order on the results page [12].

Promotion bias is more complex to investigate because it is directly entangled with position bias. As shown in (Figure 1b), promoted hotels exhibit higher click and booking rates. However, our descriptive analysis also revealed that promoted hotels appear at higher display positions on average (mean position 14.8 vs. 17.4 for non-promoted hotels). This raises the question of whether users genuinely prefer promoted hotels, or simply book them because they appear first. To disentangle this confounding effect, we applied a conditional logistic regression grouped by search ID. This isolates the within-search competition between hotels for a single booking and eliminates search-level confounding. We converted raw `price_usd` to within-search percentile ranks (`price_rel`) to ensure comparability across searches and included hotel-level quality signals: highest star rating in search (`is_best_star`), highest review score (`is_best_review`), cheapest hotel in search (`is_cheapest_in_srch`), and brand status (`prop_brand_bool`).

Without any controls, promoted hotels showed 115.1% higher booking odds ($OR = 2.151$, $p < 0.001$). After controlling for hotel quality signals alone, this advantage reduced to 95.1% ($OR = 1.951$, $p < 0.001$). This means observable quality characteristics explained only 9.3% of the raw promotion effect, proving that promotion is not simply a proxy for better hotels; if it were, controlling for quality would have neutralized the effect. Crucially, once log-transformed position was added as a control, following Craswell et al.’s observation that examination probability decays logarithmically with position [12], the effect collapsed to just 11.9% ($OR = 1.119$, 95% CI [1.083, 1.156]). Position alone accounted for

42.6% of the raw within-search promotion effect. This confirms position as the strongest single driver of the observed promotion advantage, directly reflecting a compounding mechanism: promoted hotels were historically displayed higher, accumulating bookings driven by placement rather than genuine preference, which the model then absorbed as a raw promotion-booking association.

A robustness check restricted to random-sort searches, where display position carries no information about hotel quality and cannot confound the promotion-booking association, showed a higher adjusted estimate ($OR = 1.729$, 95% CI [1.508, 1.982]). This suggests that the log-linear position assumption in the full model may not fully capture the steep, nonlinear drop in booking rates at the very top positions, leaving residual confounding. Taken together, both estimates confirm that promotion carries a small but genuine independent signal beyond quality and position, with the true position-adjusted effect lying between $OR = 1.119$ and $OR = 1.729$. This structural link between position and promotion bias makes mitigation non-trivial: a model correcting for promotion without accounting for position will inevitably learn a biased signal.

5.2 Mitigation Strategies and Limitations

We approached these two biases differently across our models, initially utilizing KNN to analyze bias reduction for promotions. An evaluation of the baseline KNN model results showed that the model assigned promoted hotels 3.5 rank positions higher on average within the same search (Cohen’s $d = -0.69$, 95% CI [-3.534, -3.423]). This exceeds what the position-adjusted promotion effect would justify, indicating the model learned the raw promotion booking association directly from the training data. This is a structural limitation of KNN: its pointwise architecture has no concept of ranking order and cannot natively account for position bias [19]. The available remedy for KNN we found was a post-processing score adjustment, which we thought could partially offset the bias by adding a small statistical bonus to non-promoted hotels. However, our solution struggled with the severe class imbalance from the data. Because KNN estimates booking probability by aggregating labels among the k nearest neighbors, it assigns an absolute zero probability to any hotel whose neighbourhood contains no bookings. Given the $\approx 2.8\%$ booking rate, this produces a heavily zero-inflated score distribution [10]. Applying a flat bonus to non-promoted hotels, therefore skewed the distribution and introduced overprediction for non-promoted properties. This led us to consider a more structurally sound correction using the LightGBM LambdaMART model.

Unlike KNN, LightGBM’s learning-to-rank framework can incorporate positional data directly during training through inverse propensity [20]. In practice, we passed a dedicated position feature to the LightGBM ranker at training time: for each hotel observation, this feature encodes the display position at which it was historically shown in Expedia’s search results. LightGBM uses these positions to estimate how likely a hotel at each rank was to receive a booking purely due to its placement, and lowers the weight of the gradient signal from observations at top positions accordingly. The result is a model that treats position-

driven bookings as less informative and is pushed towards learning intrinsic hotel quality instead.

Applying the optimal configuration from the modelling phase (learning rate = 0.05, `num_leaves` = 255, `min_child_samples` = 50), we evaluated the correction by stratifying hotels into four display position buckets that reflect the natural shape of the booking rate curve in Figure 1c: Top (1–3), Mid-high (4–10), Mid-low (11–20), and Low (21+). A biased model will systematically assign inflated scores to hotels simply because they were shown at the top. The baseline model results confirmed this: the mean score gap between the Top and Low buckets was +1.193. After correction, this narrowed to +0.739, indicating the corrected model is considerably less sensitive to where a hotel was historically placed (Figure 4b, left panel).

The NDCG@5 analysis (Figure 4b, right panel) by position bucket tells the same story from a ranking quality perspective. In the baseline, searches where the booked hotel appeared in the Top (1–3) bucket achieved a mean NDCG@5 of 0.660: the model ranked these hotels well largely because it had learned that top-displayed hotels tend to be booked. After correction, this dropped to 0.605. The improvement is most visible at the bottom of the results: the Low (21+) bucket improved from NDCG@5 = 0.124 to 0.174, and the Mid-low (11–20) bucket from 0.183 to 0.230. The Mid-high (4–10) bucket remained largely stable (0.358 to 0.364). Taken together, these results suggest the model has partially unlearned layout position as a proxy and started ranking hotels more on their own merit. This fairness improvement comes at a moderate cost to overall performance, with NDCG@5 falling slightly from 0.4108 to 0.4026, which we could expect since some of the baseline’s predictive signal was driven by the bias.

Finally, we evaluated the correction’s impact on promotion bias specifically (Figure 4a). The baseline LightGBM ranked promoted hotels 4.948 positions higher on average within the same search (Cohen’s $d = -1.017$, large effect, 95% CI [-5.003, -4.894]), a larger gap than KNN’s 3.5 positions, which is consistent with LightGBM being the stronger model and therefore more capable of exploiting the biased training signal. After correction, this narrowed to 4.015 positions, a structural reduction that KNN could not achieve by design. The remaining gap is not a failure of the correction: given that our conditional logistic regression established a genuine independent promotion effect of between OR = 1.119 and OR = 1.729, some residual advantage for promoted hotels is expected and appropriate in a well-calibrated model.

6 Learning Outcomes

Beyond the technical skills, this project provided a steep, realistic look at what it takes to build recommendation systems under real-world constraints. It taught us how to work with an industry Kaggle dataset, which introduced problems that academic datasets rarely have: severe class imbalance, sparse booking signals, and heavy computational demands. Additionally, as this dataset was computationally demanding, we needed to learn how to use the university’s computing

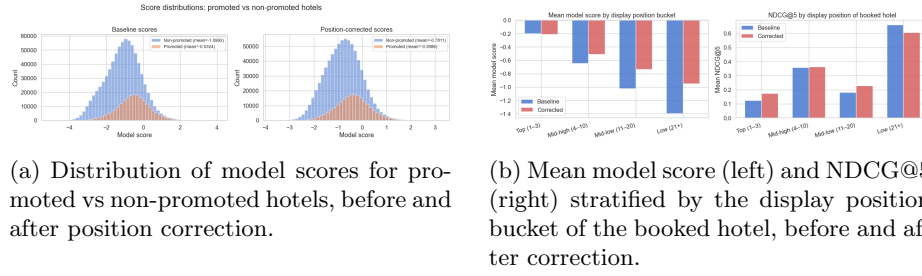


Fig. 4: LightGBM position bias correction: score distributions and position bucket analysis. The model score is the raw relevance score assigned by LightGBM to each hotel within a search: higher scores indicate a stronger predicted preference.

infrastructure, which gave us practical insight into working with large data. Moreover, taking part in the Kaggle competition provided an engaging teaching environment that motivated us to continuously improve our model.

Furthermore, we gained a clear, hands-on understanding of two distinct approaches to recommendation systems, KNN and LightGBM. We learned to identify their structural boundaries, such as how KNN’s zero-inflated score distribution resists post-processing fixes, and how position bias absorbed during training cannot be fixed without changing the architecture. A significant portion of the project relied on translating domain knowledge into informative features. We had to learn to deeply understand the dataset instead of feeding the raw data into the model. We paired this with academic methodologies, adapting textbook concepts to fit a real-world dataset, such as conditional logistic regression for within-search decisions and statistical methods to report effect size (Bootstrap CI and Cohen’s d). As part of the bias analysis, we had to iteratively try different techniques to find prejudice in our model, because a simple analysis of just grouping by one variable was ineffective. Additionally, this analysis prompted us to reflect on the real-world deployment and implications of unfair algorithms, thus preparing us for model development in the industry. Moreover, communicating these technical findings in writing and during the interview helped us practice the crucial industry skill of translating complex modeling decisions into insights for the business.

7 Conclusion

In conclusion, our project successfully addressed the Expedia hotel ranking problem by transitioning from a pointwise k-Nearest Neighbour baseline to a listwise LightGBM LambdaMART ranker, which achieved a high validation score of 0.4106. By incorporating explicit positional data during training, the final model successfully unlearned historical layout bias to rank properties based on their actual merit. Ultimately, the process demonstrated the critical value of domain-specific feature engineering and structural bias mitigation when implementing recommendation systems under real-world data constraints.

References

1. Personalize Expedia Hotel Searches - ICDM 2013, <https://kaggle.com/expedia-personalized-sort>, last accessed 2026/05/24.
2. Sarwar, B., Karypis, G., Konstan, J., Riedl, J.: Item-based collaborative filtering recommendation algorithms. In: Proceedings of the 10th international conference on World Wide Web. pp. 285–295. Association for Computing Machinery, New York, NY, USA (2001). <https://doi.org/10.1145/371920.372071>.
3. Liu, X., Xu, B., Zhang, Y., Yan, Q., Pang, L., Li, Q., Sun, H., Wang, B.: Combination of Diverse Ranking Models for Personalized Expedia Hotel Searches, <https://arxiv.org/abs/1311.7679v1>, last accessed 2026/05/07.
4. Karakasoglu, I.: Expedia Hotel Recommendations. Stanford University course project (2016), <https://kaggle.com/c/expedia-hotel-recommendations>
5. Mavalankar, A.A., Gupta, A., Gandotra, C., Misra, R.: Hotel Recommendation System. University of California San Diego (2019). <https://arxiv.org/abs/1908.07498>
6. Xie, K.L., Lee, Y.J.: Hotels at fingertips: informational cues in consumer conversion from search, click-through, to book. *Journal of Hospitality and Tourism Technology*. 9, 2, 131–147 (2018). <https://doi.org/10.1108/JHTT-03-2017-0026>,
7. Sharma, Y., Bhatt, J., Magon, R.: A Multi Criteria Review-Based Hotel Recommendation System. In: 2015 IEEE International Conference on Computer and Information Technology; Ubiquitous Computing and Communications; Dependable, Autonomic and Secure Computing; Pervasive Intelligence and Computing. pp. 687–691. IEEE (2015). <https://doi.org/10.1109/CIT/IUCC/DASC/PICOM.2015.99>
8. Forhad, M.S.A., Arefin, M.S., Kayes, A.S.M., Ahmed, K., Chowdhury, M.J.M., Kumara, I.: An Effective Hotel Recommendation System through Processing Heterogeneous Data. *Electronics*. 10, 1920 (2021). <https://doi.org/10.3390/electronics10161920>
9. Sullivan, G.M., Feinn, R.: Using Effect Size—or Why the P Value Is Not Enough. *J Grad Med Educ*. 4, 279–282 (2012). <https://doi.org/10.4300/JGME-D-12-00156.1>.
10. He, H., Garcia, E.A.: Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*. 21, 1263–1284 (2009). <https://doi.org/10.1109/TKDE.2008.239>.
11. Cover, T., Hart, P.: Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*. 13, 21–27 (1967). <https://doi.org/10.1109/TIT.1967.1053964>.
12. Craswell, N., Zoeter, O., Taylor, M., Ramsey, B.: An experimental comparison of click position-bias models. In: Proceedings of the 2008 International Conference on Web Search and Data Mining. pp. 87–94. Association for Computing Machinery, New York, NY, USA (2008). <https://doi.org/10.1145/1341531.1341545>.
13. Lipton, Z.C.: The Mythos of Model Interpretability, <https://arxiv.org/abs/1606.03490v3>, last accessed 2026/05/22.
14. McFadden, D.: Conditional Logit Analysis of Qualitative Choice Behavior. (1972).
15. Adam, Hamner, B., Friedman, D., SSA Expedia.: Personalize Expedia Hotel Searches - ICDM 2013. Kaggle (2013). <https://kaggle.com/competitions/expedia-personalized-sort>
16. Xu, B., MLRush., BrickMover.: Bing Xu / MLRush / BrickMover — My Approach. Kaggle (2013). <https://www.kaggle.com/competitions/expedia-personalized-sort/writeups/bing-xu-mlrush-brickmover-my-approach>, last accessed 2026/05/07
17. Evidently AI Team.: Normalized Discounted Cumulative Gain (NDCG) Explained. Evidently AI (2025). <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>, last accessed 2026/05/07

18. Aggarwal, C.C., Yu, P.S.: Outlier detection for high dimensional data. In: Proceedings of the 2001 ACM SIGMOD international conference on Management of data. pp. 37–46. Association for Computing Machinery, New York, NY, USA (2001). <https://doi.org/10.1145/375663.375668>.
19. Liu, T.-Y.: The Pointwise Approach. In: Liu, T.-Y. (ed.) Learning to Rank for Information Retrieval. pp. 33–47. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-14267-3_2.
20. LightGBM Development Team.: LGBMRanker API Documentation. LightGBM (2025). <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMRanker.html>, last accessed 2026/05/07
21. Ng, A.Y.: Feature Selection, L1 vs. L2 Regularization, and Rotational Invariance. Proceedings of the 21st International Conference on Machine Learning, Banff, Canada (2004). <https://icml.cc/Conferences/2004/proceedings/papers/354.pdf>, last accessed 2026/05/07
22. Murel, J., Kavlakoglu, E.: What Is Regularization? IBM (2023). <https://www.ibm.com/think/topics/regularization>, last accessed 2026/05/07.
23. Burges, C.J.C.: From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82 (2010). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf>, last accessed 2026/05/07
24. Murrell, T.: LambdaMART Explained: The Workhorse of Learning-to-Rank. Shaped AI (2025). <https://www.shaped.ai/blog/lambdamart-explained-the-workhorse-of-learning-to-rank>, last accessed 2026/05/07